Guía de Actividades Práctico-Experimentales Nro. 002

1. Datos Generales

Asignatura	Estructura de Datos
Integrantes	- Freddy Ordoñez - Yimmy Angulo - Richard Guaman - Erick Jahirs - Joaquin Moscol
Ciclo	3ro A / B
Unidad	Estructuras de datos estáticas (arreglos, conjuntos, registros, archivos) y estructuras de datos dinámicas (listas, pilas y colas)
Resultado de aprendizaje de la unidad	R1. Implementa algoritmos simples de búsqueda y explica las diferencias en el orden de complejidad computacional, bajo los principios de solidaridad,
Práctica Nro.	02
Título de la Práctica	Implementación de acceso secuencial y aleatorio a archivos con métricas de I/O
Nombre del Docente	Cristian Ramiro Narváez Guillén
Fecha	jueves 8 de octubre 2026 - viernes 9 de octubre 2026.
Horario	jueves 8 de 08:30 – 09:30 viernes 9 de 11:30 – 13h30
Lugar	EVA Aula
Tiempo planificado en el Sílabo	3 horas

2. Objetivo(s) de la Práctica:

Implementar algoritmos de acceso secuencial para la persistencia masiva de datos en disco mediante el módulo nativo de I/O en JavaScript.

Cuantificar empíricamente la latencia computacional (tiempo de respuesta $T(n)$) y el impacto en la memoria Heap ($S(n)$) al procesar archivos de forma síncrona (bloqueante) versus flujos asíncronos (*Streams*).

Evaluar las decisiones de diseño físico en el acceso a almacenamiento mecánico o de estado sólido bajo los principios de *Green Computing*, analizando cómo la optimización de los accesos I/O reduce el consumo energético del hardware.

3. Materiales y reactivos:

- Software Node.js (versión LTS 20.x o superior) instalado en el sistema.
- IDE moderno para desarrollo (Visual Studio Code, WebStorm).
- Conexión a Internet estable para consultas a la documentación técnica oficial de Node.js (módulo fs).
- Acceso a la consola del sistema (Terminal de Linux/macOS o PowerShell en Windows).
- Recursos Digitales: Sílabo de la asignatura, repositorio base en GitHub.
- Plataforma EVA UNL.

4. Equipos y herramientas

Recursos Físicos

- Aula de Computación.

Computadora personal o de laboratorio:

- Sistema operativo: Windows, Linux o macOS.
- Procesador: Mínimo 2 núcleos (recomendado 4).
- Memoria RAM: Mínimo 4 GB.
- Almacenamiento: Disco HDD o SSD con al menos 1 GB de espacio libre temporal.

5. Procedimiento / Metodología.

Enfoque PID-2025:

- **ABP (Aprendizaje Basado en Problemas):** El equipo satelital del APE 001 ahora necesita leer el millón de coordenadas ($N = 10^6$) que han sido transmitidas y almacenadas en un archivo físico en el servidor. El problema radica en que leer el archivo de una sola vez colapsa la memoria RAM del contenedor en la nube. Se debe diseñar una estrategia para leer los datos con el mínimo impacto temporal y espacial.
- **ABI (Aprendizaje Basado en Investigación):** El estudiante investigará cómo interactúa el *Event Loop* de JavaScript (motor V8 y libuv) con las llamadas del sistema operativo (Syscalls) para el acceso a disco.
- **Modalidad**: Individual y análisis comparativo grupal posterior.

Tarea 1: Generación del Conjunto de Datos Masivo (Mock Data)

Para poder medir el I/O, primero necesitamos un archivo físico considerable.

1. **En el directorio de su proyecto, cree un archivo llamado lab02_generador.js.**
2. **Implemente el siguiente código para escribir un archivo CSV con 10^6 registros utilizando escritura en flujo (Stream) para no saturar la memoria durante su creación.**

```
const fs = require('fs');
const { performance } = require('perf_hooks');

const FILE_NAME = 'coordenadas_masivas.csv';
const N = 1000000;

console.log(`[Generador] Iniciando escritura de ${N} registros...`);
const start = performance.now();

// Utilizamos un Stream de escritura (Write Stream)
const writableStream = fs.createWriteStream(FILE_NAME);
writableStream.write('id,latitud,longitud\n'); // Cabecera

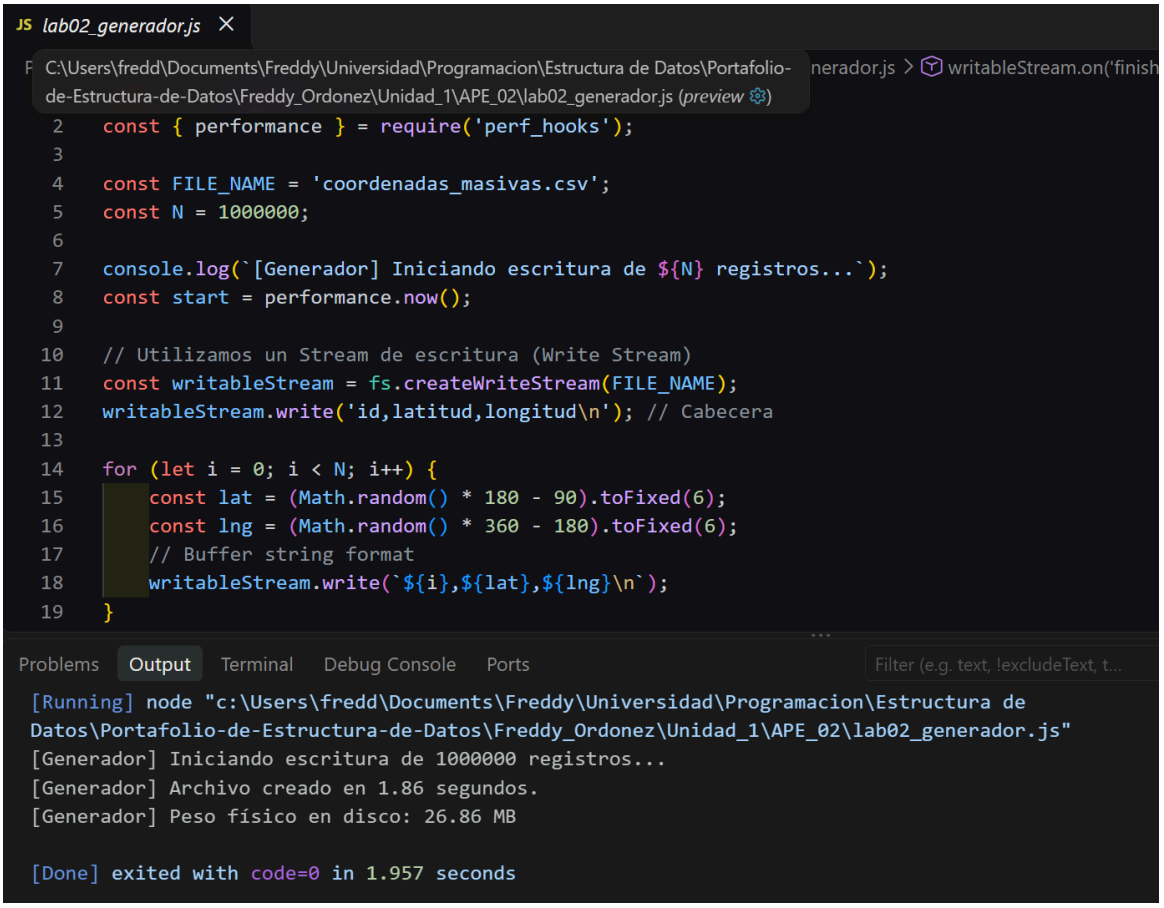
for (let i = 0; i < N; i++) {
   const lat = (Math.random() * 180 - 90).toFixed(6);
   const lng = (Math.random() * 360 - 180).toFixed(6);
   // Buffer string format
   writableStream.write(`${i},${lat},${lng}\n`);
}

writableStream.end();

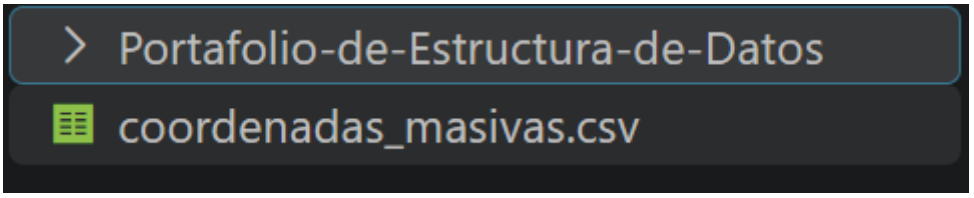
writableStream.on('finish', () => {
   const end = performance.now();
       console.log(`[Generador] Archivo creado en ${((end - start) / 1000).toFixed(2)} segundos.`);
     // Calcular tamaño del archivo en disco
   const stats = fs.statSync(FILE_NAME);
   const sizeInMB = stats.size / (1024 * 1024);
```

```
    console.log(`[Generador] Peso físico en disco: ${sizeInMB.toFixed(2)} MB`);
});
```

3. ***Ejecute en terminal: node lab02_generador.js y verifique la creación del archivo coordenadas_masivas.csv.***



Verificación que se creó el archivo:



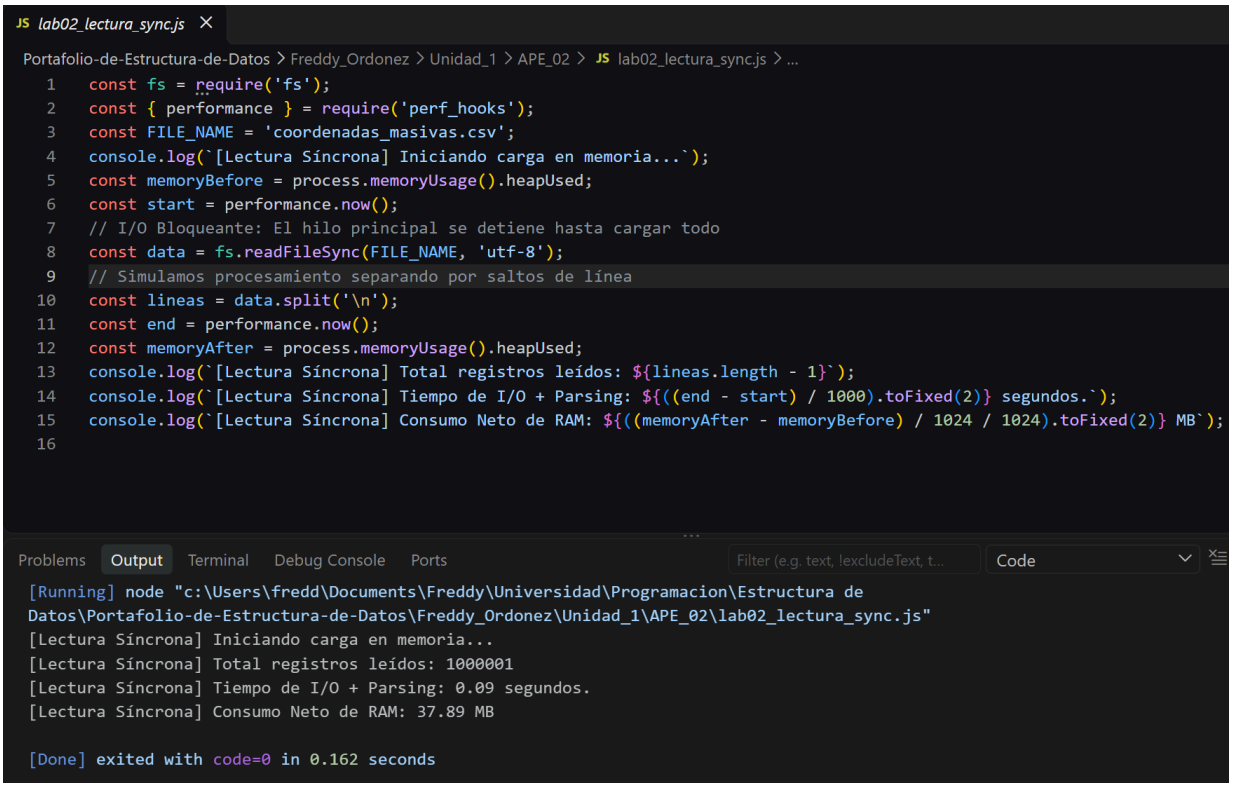
Tarea 2: Enfoque 1 - Lectura Síncrona (Bloqueante / Espacial)

Este enfoque representa la manera más intuitiva, pero menos eficiente, de procesar un archivo físico.

1. **Cree un archivo llamado lab02_lectura_sync.js.**
2. **Implemente el código para cargar todo el archivo en la RAM.**

```
const fs = require('fs');
const { performance } = require('perf_hooks');
const FILE_NAME = 'coordenadas_masivas.csv';
console.log(`[Lectura Síncrona] Iniciando carga en memoria...`);
const memoryBefore = process.memoryUsage().heapUsed;
const start = performance.now();
// I/O Bloqueante: El hilo principal se detiene hasta cargar todo
const data = fs.readFileSync(FILE_NAME, 'utf-8');
// Simulamos procesamiento separando por saltos de línea
const lineas = data.split('\n');
const end = performance.now();
const memoryAfter = process.memoryUsage().heapUsed;
console.log(`[Lectura Síncrona] Total registros leídos: ${lineas.length - 1}`);
console.log(`[Lectura Síncrona] Tiempo de I/O + Parsing: ${((end - start) / 1000).toFixed(2)} segundos.`);
console.log(`[Lectura Síncrona] Consumo Neto de RAM: ${((memoryAfter - memoryBefore) / 1024 / 1024).toFixed(2)} MB`);
```

3. ***Ejecute node lab02_lectura_sync.js y tabule el tiempo y el impacto en memoria.***



Tabulación del tiempo y el impacto en memoria:

Métrica	*Valor Registrado*
Total de registros leídos	*1,000,001*
Tiempo de I/O + Parsing	*0.09 segundos*
Consumo Neto de RAM (Heap)	*37.89 MB*
Tiempo total de ejecución (Node)	*0.162 segundos*

Tarea 3: Enfoque 2 - Lectura Asíncrona con Streams ($O(1)$ Espacial)

1. Cree el archivo lab02_lectura_stream.js.
2. Implemente la abstracción de flujos (Streams) para leer fragmentos (*chunks*) secuenciales, reduciendo la complejidad espacial teórica de $S(n) = n \cdot \text{bytes}$ a un constante $S(n) = O(1)$ correspondiente al tamaño del *buffer*.

```
const fs = require('fs');
const { performance } = require('perf_hooks');
const FILE_NAME = 'coordenadas_masivas.csv';
console.log(`[Lectura por Stream] Iniciando procesamiento...`);
const memoryBefore = process.memoryUsage().heapUsed;
const start = performance.now();
let totalRegistros = 0;
// Creamos un flujo de lectura secuencial no bloqueante
const readableStream = fs.createReadStream(FILE_NAME, { encoding: 'utf-8' });
readableStream.on('data', (chunk) => {
    // Procesamos el chunk de datos actual sin almacenarlo completo
    // Contamos cuántos saltos de línea hay en este fragmento
    let lineBreakCount = (chunk.match(/\n/g) || []).length;
    totalRegistros += lineBreakCount;
```

```
});
readableStream.on('end', () => {
    const end = performance.now();
    const memoryAfter = process.memoryUsage().heapUsed;
        console.log(`[Lectura por Stream] Total registros procesados: ${totalRegistros}`);
    console.log(`[Lectura por Stream] Tiempo de I/O parcializado: ${((end - start) / 1000).toFixed(2)} segundos.`);
    console.log(`[Lectura por Stream] Consumo Neto de RAM: ${((memoryAfter - memoryBefore) / 1024 / 1024).toFixed(2)} MB`);
});
```

3. ***Ejecute node lab02_lectura_stream.js. Compare drásticamente la métrica del consumo neto de RAM entre la Tarea 2 y la Tarea 3.***

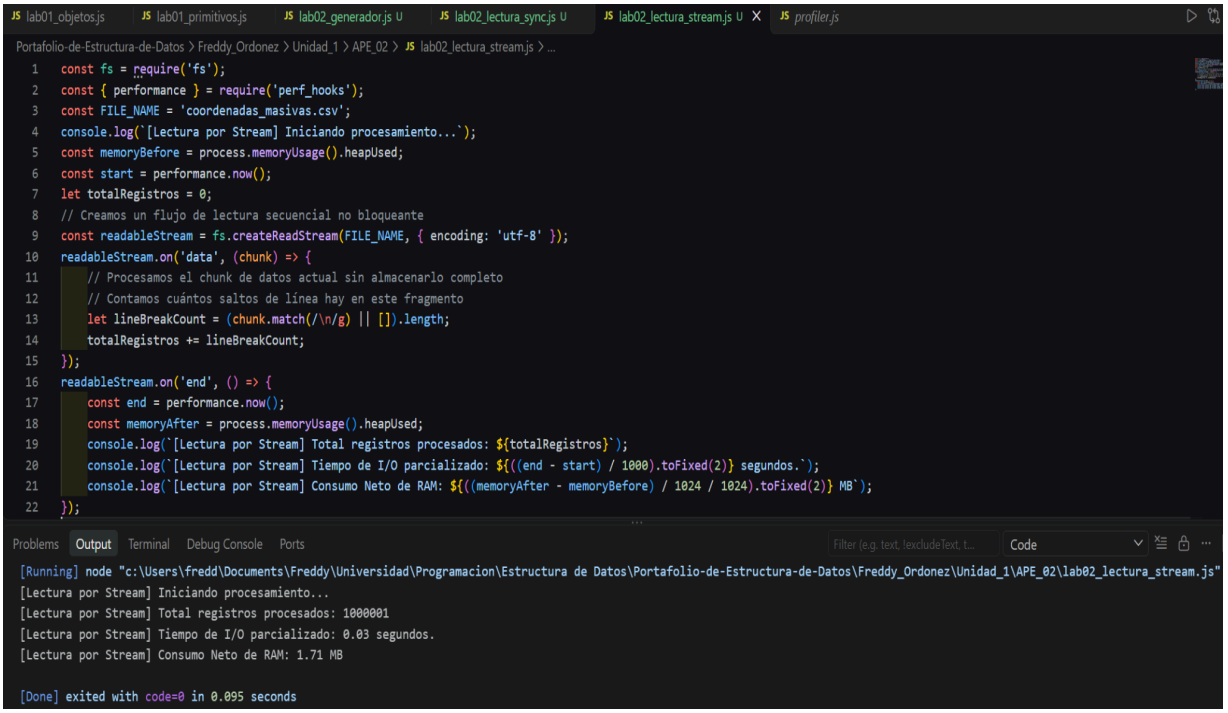


Tabla Comparativa de Rendimiento: I/O Síncrono vs. Streams

Métrica Evaluada	**Enfoque 1: Lectura Síncrona (lab02_lectura_sync.js)**	**Enfoque 2: Lectura por Streams (lab02_lectura_stream.js)**
Total de Registros	1,000,001	1,000,001

Tiempo de I/O / Procesamiento	0.09 segundos	0.04 segundos
Consumo Neto de RAM	37.89 MB	1.71 MB
Complejidad Espacial	$O(n)$ **(Carga masiva en memoria RAM)**	$O(1)$ **(Huella de memoria constante por *buffer*)**
Comportamiento de Operación	Bloqueante detiene la ejecución principal	No bloqueante asíncrono por fragmentos

Tarea 4: Análisis Matemático de Rendimiento (ABI)

Basado en los resultados empíricos, los estudiantes investigarán el diseño del bus de datos y el *Thread Pool* interno (libuv) que gestiona estas llamadas.

- **Demuestre teóricamente (utilizando notación de orden asintótico) por qué la función bloqueante impone** $S(n) = O(n)$ **frente al enfoque *Stream*** $S(n) = O(1)$ **.**

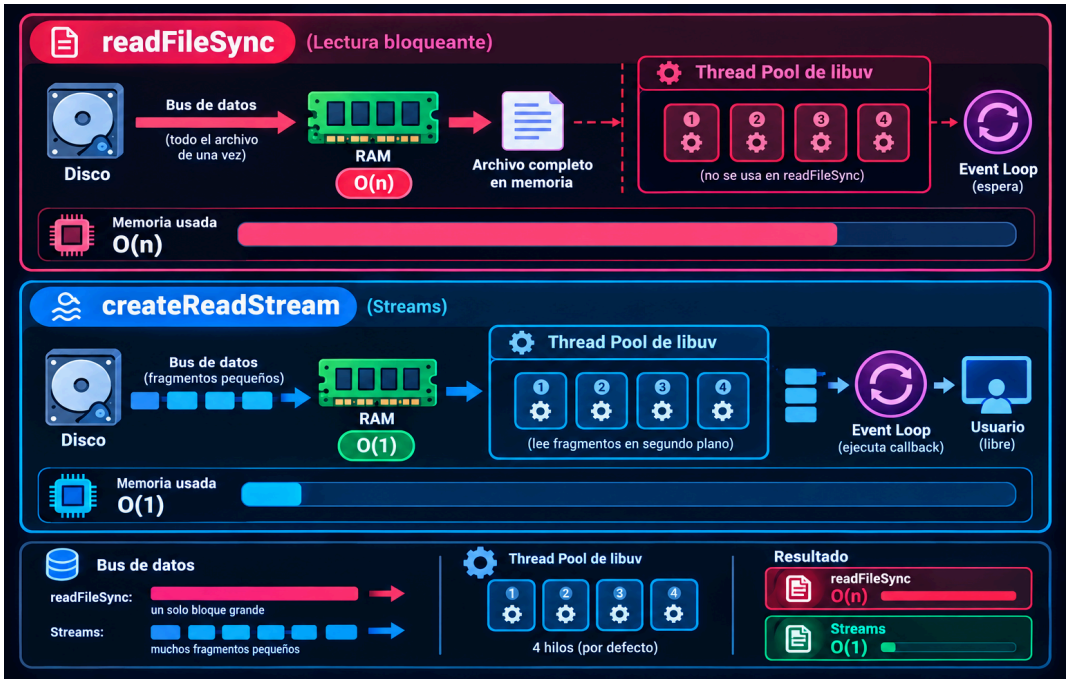
  Demostración Teórica. Sea **n** el número de registros y **r** los bytes que ocupa cada uno en memoria. ***readFileSync*** devuelve el contenido completo, así que no puede terminar hasta tener todo cargado. Además, ***split('\n')*** crea un arreglo con n elementos, por lo que la memoria usada es ***S(n) = n · r. Como S(n) ≤ c · n con c = r, se cumple que S(n) ∈ O(n).*** Con los datos medidos, ***r ≈ 39.7 bytes*** por registro, y esto significa que si el archivo llegará a 100 millones de registros, haría falta unos **3.8 GB de RAM**.

  Con Streams, el archivo se lee en fragmentos de tamaño **fijo B (64 KB por defecto)**, y en cada momento solo hay un fragmento en memoria, que el siguiente reutiliza al terminar. Por eso la memoria es ***S(n) = B + K, con K constante,*** y no depende de ***n, así que S(n) ∈ O(1).*** Al compararla con la lectura bloqueante, ***el cociente (n · r) / B tiende a***

infinito cuando n crece, lo que significa que la ventaja de Streams aumenta mientras más grande sea el archivo.

Thread Pool de libuv y bus de datos. El ***readFileSync*** ejecuta la lectura en el hilo principal, que queda detenido hasta que el sistema operativo entrega todo el archivo. En cambio, **createReadStream** pide cada fragmento a libuv, que lo envía a su Thread Pool (4 hilos por defecto). Cuando el fragmento está listo, el Event Loop ejecuta el callback, y mientras tanto el hilo principal queda libre para atender otras tareas. Por el bus de datos, la información viaja del disco a la RAM en trozos pequeños y constantes con Streams, mientras que con la lectura síncrona viaja en un solo bloque grande que ocupa la memoria de golpe.

Con este gráfico se entiende mucho mejor este proceso:



- **Estructure su respuesta relacionándola con el uso energético del servidor (menos RAM implica menor recolección de basura y menor gasto térmico del procesador).**

Relación con el consumo energético. Con menos recolección de basura, el procesador dedica menos ciclos a tareas que no aportan al problema, lo que genera menos calor y exige menos enfriamiento. En un servidor que procesa archivos grandes todo el día, esa diferencia se

traduce en menor consumo eléctrico. Por eso, además de ser más eficiente, el enfoque con Streams es más sostenible, en línea con los principios de Green Computing.

Repositorio de Github Grupal:

https://github.com/freddyordonez/Portafolio-de-Estructura-de-Datos.git

6. Resultados esperados:

- Se obtendrán métricas directas que evidencian cómo una operación aparentemente trivial de lectura/escritura puede causar cuellos de botella de I/O (*Input/Output Bottleneck*). El uso de *Streams* demostrará que la persistencia masiva puede gestionarse secuencialmente manteniendo una huella de memoria plana, logrando un código ambiental y computacionalmente sostenible.

7. Preguntas de Control:

- Desde una perspectiva filosófica profunda y analítica, responda:

1. **Causa Material:** Al realizar una lectura síncrona mediante readFileSync, ¿cuál es la "materia" que se satura en el ordenador y cómo se diferencia físicamente del estado de reposo de un archivo almacenado magnética o electrónicamente en el disco duro?

   Al realizar una lectura síncrona con ***readFileSync***, la "materia" que se satura es la memoria RAM de nuestras computadoras. Esto se da porque la instrucción obliga al sistema a cargar todo el archivo de golpe y guardarlo de un solo tirón en la memoria principal, como si intentaras meter todo el contenido de una biblioteca en una sola mesa pequeña.

   Físicamente, esto es muy distinto a cómo se guarda el archivo en el disco duro cuando está en reposo. En el disco ya sea un disco tradicional o un SSD, la información se graba de forma permanente mediante imanes o pequeños voltajes que no necesitan electricidad constante para no borrarse; en cambio, lo que sube a la RAM se convierte en energía eléctrica temporal que desaparece si apagas el equipo y satura los recursos del sistema al instante.

2. **Causa Formal:** El archivo CSV (Valores Separados por Comas) posee una "forma" específica (texto secuencial lineal). ¿De qué manera esta forma impone restricciones algorítmicas sobre nuestra capacidad para

acceder aleatoriamente a la coordenada ***N=500,000*** sin tener que recorrer las ***N=499,999*** coordenadas previas?

La forma del archivo CSV es la de un texto lineal y secuencial donde cada registro tiene una longitud variable: el id crece de 1 a 7 dígitos y la latitud y la longitud pueden llevar o no signo negativo, por lo que cada línea mide distinto. Como no existe un tamaño fijo por registro, no se puede calcular con una fórmula (`posición = i · L`) en qué byte empieza la coordenada N = 500,000. La única forma de encontrarla es leer el archivo desde el inicio contando saltos de línea, lo que cuesta O(n) y obliga a recorrer las 499,999 coordenadas previas.

En cambio, si los registros fueran de tamaño fijo (por ejemplo, en un archivo binario), se podría saltar directamente a la posición con ***`fs.read`*** indicando el offset, con costo O(1). También se podría construir una sola vez un índice con la posición de cada registro: costaría O(n) al inicio, pero luego cada acceso aleatorio sería O(1). Así, la forma del archivo es la que impone la restricción algorítmica.

3. **Causa Eficiente:** Durante el uso de *Streams* asíncronos en Node.js, el *Event Loop* delega la operación al sistema operativo. En este contexto, ¿quién actúa como el verdadero motor (causa eficiente) que transfiere los electrones desde el disco hasta la memoria RAM en fragmentos (chunks)?

   Cuando usamos Streams asíncronos, el Event Loop de Node.js funciona más bien como un coordinador que da la instrucción inicial. El verdadero motor, o "causa eficiente", que se encarga de hacer el esfuerzo físico de mover la información desde el disco duro a la memoria RAM en pequeños fragmentos (chunks) es el propio sistema operativo trabajando en conjunto con el hardware de nuestra computadora. Básicamente, en nuestro código con Node.js nosotros simplemente le delegamos esta lectura pesada al sistema operativo a través de llamadas del sistema (Syscalls). Es el sistema operativo quien toma el control real y transfiere los datos poco a poco hacia la memoria, lo que permite que nuestro programa fluya sin colapsar la RAM ni quedarse bloqueado.

4. **Causa Final:** Si todo dato en ejecución debe residir en la memoria principal para ser computado por la CPU, ¿cuál es el propósito (telos) de la "persistencia" en el disco físico, sabiendo que las operaciones de I/O añaden un costo de tiempo monumental en la escala de microsegundos (μs) frente a los nanosegundos (ns) de la memoria caché?

   La finalidad (telos) de la persistencia en el disco físico es garantizar la conservación de la información más allá del tiempo de ejecución de un programa, permitiendo que los datos sobrevivan al apagado del equipo, a los reinicios y a la finalización de los procesos.

   Aunque la memoria caché y la memoria RAM permiten acceder a los datos con mayor rapidez, son recursos orientados principalmente al procesamiento temporal. En cambio, el almacenamiento persistente permite conservar información durante largos períodos, sin depender de que el sistema permanezca encendido.

   Desde una perspectiva técnica, existe una compensación entre velocidad y permanencia: la memoria caché opera generalmente en nanosegundos, mientras que las operaciones de almacenamiento pueden requerir microsegundos o incluso milisegundos, dependiendo del dispositivo y de la operación. Sin embargo, esa diferencia de latencia no convierte al almacenamiento persistente en un recurso innecesario.

   En última instancia, la persistencia transforma el procesamiento efímero en conocimiento conservable y recuperable. Sin ella, una base de datos perdería su información al apagarse el equipo, una aplicación no podría recuperar sus configuraciones y un sistema financiero no podría conservar el historial de sus transacciones. Por tanto, la finalidad del disco no es competir con la velocidad de la memoria, sino garantizar la continuidad de la información en el tiempo.

   La arquitectura de los sistemas informáticos responde así a dos necesidades complementarias: la memoria proporciona velocidad

para calcular en el presente; la persistencia proporciona continuidad para recuperar lo producido en el pasado y utilizarlo en el futuro.

5. **Esencia y Existencia:** Un conjunto de 10^6 coordenadas existe como un bloque monolítico en el archivo data.csv, pero en el enfoque de *Streams*, solo una porción existe en la RAM a la vez. ¿Es conceptualmente el "mismo" conjunto de datos procesado, si en la memoria jamás llegó a existir de manera unificada? Argumente.

   Sí, es el mismo conjunto de datos, aunque no exista de la misma manera. Lo que define al conjunto es su contenido: un millón de coordenadas en un orden determinado. Con los dos enfoques se procesan exactamente los mismos datos y sale el mismo resultado (1,000,001 registros). Lo que cambia es cómo se presenta en la memoria: con ***readFileSync*** ocupa 37.89 MB de golpe, y con Streams solo ocupa 1.71 MB porque llega por partes.

   Es como leer un libro completo de una vez o página por página. Nunca tienes todas las páginas frente a ti al mismo tiempo, pero la historia sigue siendo la misma. En este caso el conjunto sigue completo en el disco, y el Stream lo recorre en orden hasta el final, así que no se pierde nada. Eso sí, hay un límite: con Streams solo se pueden hacer tareas que se resuelvan por partes, como contar o filtrar. Si hiciera falta ordenar todo el millón de coordenadas, sí se necesitaría tenerlo completo en la RAM.

   Por eso, que dos formas de procesar den el mismo resultado no significa que valgan lo mismo. Cargar todo en memoria gasta mucha más RAM que usar Streams, lo que implica más trabajo para el procesador y más consumo de energía. En un proyecto de computación sostenible importa tanto el resultado como lo que cuesta conseguirlo.

Repositorio de Github Grupal:

https://github.com/freddyordonez/Portafolio-de-Estructura-de-Datos.git

8. Bibliografía

- P. W. Bible and L. Moser, *An Open Guide to Data Structures and Algorithms*. PALNI Open Press, 2023. [Online]. Available: https://open.palni.org/
- Node.js Foundation, "File system | Node.js v20.x Documentation", *Node.js API*, 2023. [Online]. Available: https://nodejs.org/docs/latest-v20.x/api/fs.html
- S. Tilkov et al., *Node.js in Action*, 2nd ed. Manning Publications, 2017.